

INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG

Department of Computer Science and Engineering

CSE-2340: Software Development 1

Git Basics

Week 1 · Class 4 · init, add, commit, push, repository, .gitignore

Md Sadman Hafiz

Lecturer, Dept. of CSE, IIUC

Autumn 2026 | 50-minute laboratory class

Recall & Today's Outcome

? Recall from Class 3

- Which tag holds the main content of a page?
- What does the alt attribute do?
- How do we link a label to an input?
- Show me your profile folder on screen.

✓ By the end of today you can

- Explain what version control solves
- Tell the difference between Git and GitHub
- Run init, status, add, commit
- Create a GitHub repository
- Push your profile page to GitHub
- Write a .gitignore file

The Problem: Why Version Control?

How students save work today

- project.html
- project_final.html
- project_final2.html
- project_final_REAL_last.html

What goes wrong

- Which file is actually the newest?
- Yesterday's working code is gone
- Two friends edit and overwrite each other
- The laptop dies and so does the project

Git is a save point system for your code

- Analogy: a video game. You save before the boss fight and you can always load that save.
- Every commit is one save point with a name, a time, and the exact state of every file.
- You can go back to any save point, and you can see exactly what changed between two of them.

Git vs GitHub

Git

- A program on YOUR computer
- Works fully offline
- Records the history of your files
- Free and open source



push

GitHub

- A website that stores repositories
- Needs the internet
- Your online backup and portfolio
- Where teams share code

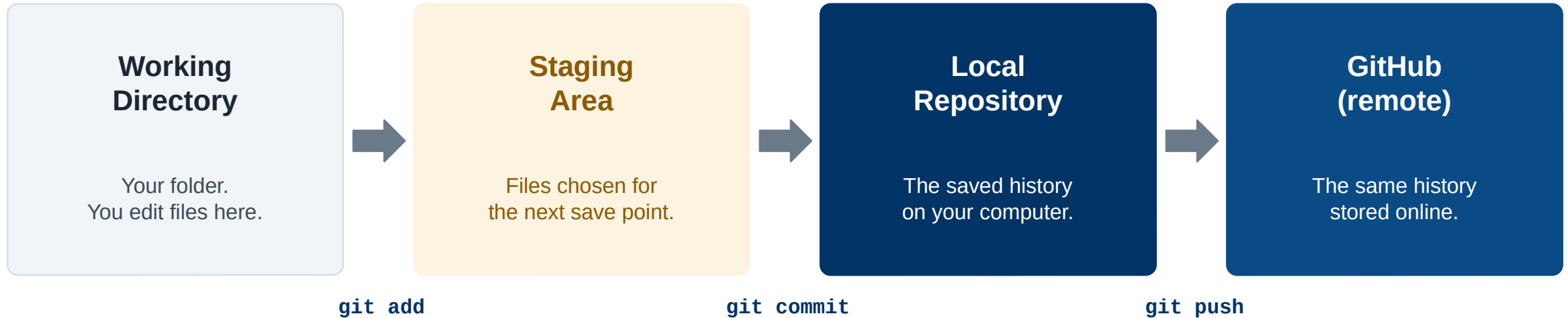
Analogy

- Git is the camera that takes the photos. GitHub is the online album where you upload them.
- You can use Git with no GitHub account. You cannot upload anything to GitHub without Git (or the website upload button).
- In this course every mini project is submitted as a GitHub repository link plus a deployed URL.

CONCEPT

The Three Areas of Git

Every change travels through the same three places. Learn this diagram and Git stops being confusing.



```
# The whole course workflow in four lines
git add .                # working directory -> staging area
git commit -m "message"  # staging area      -> local repository
git push                 # local repository -> GitHub
```

Install Git and Configure It Once

```
# 1. Check that Git is installed
git --version

# 2. Tell Git who you are (do this ONCE)
git config --global user.name "Md Sadman"
git config --global user.email "sadman@example.com"

# 3. Check what was saved
git config --list
```

Where to get Git

- git-scm.com/downloads
- Windows: install Git Bash
- Accept the default options

! Use your real email

- Use the same email as your GitHub account
- Your name appears on every commit forever
- --global means: for all my projects

01

Live Build: Folder to GitHub

Open your Class 3 profile folder. We do every command together.

Command 1: git init

```
cd my-profile  
git init
```

```
Initialized empty Git repository in  
/Users/sadman/my-profile/.git/
```



See the hidden folder

- Windows: dir /a
- Mac / Linux: ls -a
- VS Code hides it by default
- Never edit anything inside .git by hand

What just happened

- A hidden folder .git was created
- It stores the whole history
- Delete .git and the history is gone
- Run this ONCE per project

Common mistake

- Running git init in the wrong folder
- Running git init inside Documents or Desktop
- Fix: check with pwd or cd first

Command 2: git status

Your most used command. Run it after everything. It always tells you what Git is thinking.

```
git status
```

```
On branch main
```

```
No commits yet
```

```
Untracked files:
```

```
  (use "git add <file>..." to include)
```

```
    index.html
```

```
    images/
```

```
nothing added to commit but untracked  
files present
```

The three words to know

- Untracked - Git has never seen this file
- Staged - ready for the next commit
- Modified - changed since the last commit



Good habit

- Run git status before add
- Run git status after add
- Read the message - it tells you the next command

Command 3: git add

```
# Stage one file
git add index.html

# Stage everything in this folder
git add .

git status

Changes to be committed:
  new file:   index.html
  new file:   images/profile.jpg
```



Analogy: the shopping basket

- Your folder is the whole shop
- git add puts items in your basket
- git commit is paying at the counter
- Nothing is saved until you commit

add filename or add . ?

- git add index.html - one file only
- git add . - every changed file
- Use add . in this course
- Use one file when you want separate commits

Command 4: git commit

```
git commit -m "Add semantic profile page"
```

```
git log --oneline
```

```
[main (root-commit) 8f3c1a9]  
Add semantic profile page  
2 files changed, 48 insertions(+)
```

Rules for -m

- -m means message. Without it Git opens an editor
- Say WHAT changed, in present tense
- One commit = one meaningful change
- Mini projects are marked partly on commit history



Good commit messages

- Add contact form to profile page
- Fix broken image path
- Add skills and education sections

Bad commit messages

- update
- asdf
- final final
- changes

Command 5: Connect to GitHub

On the GitHub website

- Click New repository
- Name it: my-profile
- Keep it Public
- Do NOT tick Add a README
- Click Create repository



Why no README yet

- A README on GitHub creates a commit there
- Your computer does not have that commit
- The push is then rejected
- Start empty - it is simpler

```
# Copy the HTTPS URL GitHub shows you, then:
git remote add origin https://github.com/YOUR-USERNAME/my-profile.git

git branch -M main      # name the branch main
git remote -v           # check the connection
```

Command 6: git push

```
# First push of a new repository  
git push -u origin main
```

```
# Every push after that  
git push
```

```
Enumerating objects: 6, done.  
Writing objects: 100% (6/6), 1.21 KiB  
To https://github.com/sadman/my-profile.git  
* [new branch]      main -> main
```



Success check

- Refresh the GitHub page
- Your index.html is there
- This link is what you submit for mini projects

What -u does

- Links your local main to origin main
- Needed only on the first push
- After that git push is enough



Login: use a token

- GitHub does not accept your password
- Settings > Developer settings > Personal access tokens
- Generate a classic token, tick repo
- Paste the token where it asks for a password

The .gitignore File

Some files must never go to GitHub. List them in a file named .gitignore in the project root.

```
# Dependencies
node_modules/

# Secret keys and passwords
.env

# Operating system files
.DS_Store
Thumbs.db

# Editor settings
.vscode/
.idea/
```

Never push these

- API keys and passwords (.env)
- node_modules - thousands of files
- Personal editor settings

How to write it

- One rule per line
- folder/ ignores a whole folder
- *.log ignores every .log file
- Create it BEFORE your first commit

The Full Workflow, Start to Finish

```
# ----- ONCE PER PROJECT -----  
cd my-profile  
git init  
git add .  
git commit -m "Initial commit: semantic profile page"  
git branch -M main  
git remote add origin https://github.com/YOUR-USERNAME/my-profile.git  
git push -u origin main  
  
# ----- EVERY TIME YOU CHANGE SOMETHING -----  
# 1. edit your files in VS Code and save  
git status          # what changed?  
git add .           # stage the changes  
git commit -m "Add hobbies section"  
git push            # send to GitHub
```

Memorise the daily loop: status → add → commit → push. You will run it hundreds of times this semester.

Common Errors and Fixes

fatal: not a git repository

You are in the wrong folder. cd into the project, or you forgot git init.

Authentication failed

You typed your GitHub password. Use a personal access token instead.

nothing to commit

You forgot git add. Stage first, then commit.

Updates were rejected

GitHub has a commit you do not have (usually a README). Run git pull origin main --rebase then push.

src refspec main does not match

You have not made any commit yet, or your branch is called master. Run git branch -M main.

Merge conflicts exist, but we do not cover them today. We handle branches and pull requests in Week 8.

Assignment - Class 4

1 Part 1 - In-class demonstration

- Time: last 5 minutes of this class
- Task: push your Class 3 profile page to a public GitHub repository named my-profile
- Add a .gitignore with at least three rules
- Show the live GitHub page on your screen
- Submission: show the repository URL to the instructor before leaving
- Marks: 2 (checked on the spot)

2 Part 2 - Home task

- Write a README.md describing your page (use the template given)
- Make at least three more commits with clear messages, for example: add hobbies section, fix image path, add README
- Push everything to GitHub
- Deadline: before the next class (Week 2, Class 1)
- Submission: paste your repository URL in the class group / LMS
- Marks: 3

Every command from today is in the file git-cheatsheet.md. Keep it open while you work.

48-50 MINUTES

Marking Rubric & Exit Check

RUBRIC - 5 MARKS TOTAL

Repository created and code pushed to GitHub	1.5
At least four commits with meaningful messages	1.5
Working .gitignore with sensible rules	1.0
README.md describes the project clearly	1.0

Exit question 1 - concept

What is the difference between git add and git commit?

Exit question 2 - practical

Write the exact command that saves your staged changes with the message "Add footer".

Next class (Week 2, Class 1): navigation, forms and tables - practical HTML patterns.